THE ODIN PROJECT · JAVASCRIPT COURSE

Organizing Your JavaScript Code



Aykhan Dadashov · Web Technologies · 28.10.2026

01 What I covered

1
Introduction: How This Course Will Work

2
Organizing Your JavaScript Code

3
Organizing Code with Objects

4
Object Constructors

5
Project: Library

6
Factory Functions and the Module Pattern

7
Project: Tic Tac Toe

02

Introduction: how the course works

Builds on Foundations – we already know variables, functions, loops and the DOM

Focus on structure – not only "does it work", but how to keep code clean when the project grows

Read, then build – each lesson has articles and knowledge checks, and every few lessons there is a project

Ask for help – the course recommends the Odin community on Discord when you are stuck

14 lessons in "Organizing Your JavaScript Code"

4 projects in the section: Library, Tic Tac Toe, Restaurant Page, Todo List

2 projects done so far: Library and Tic Tac Toe

03 Organizing code with objects

Object literal – groups data that belongs together in one place

Dot or bracket notation – player.name or player["name"] (brackets work with variables)

Methods – functions inside an object, so data and behaviour live together

this – inside a method it points to the object that called it

Result – instead of many separate variables, one object keeps the code organized

objects.js

```
// without objects: loose variables
const playerOneName = "Aykhan";
const playerOneMarker = "X";

// with an object: grouped together
const playerOne = {
  name: "Aykhan",
  marker: "X",
  sayHello() {
    return `Hi, I am ${this.name}`;
  },
};

playerOne.name;         // "Aykhan"
playerOne["marker"];    // "X"
playerOne.sayHello();   // "Hi, I am Aykhan"
```

04 Object constructors and prototypes

Constructor – a normal function we call with new to create many similar objects

new – creates an empty object, sets this to it and returns it

Prototype – every object has one; methods there are shared, not copied

Prototype chain – if a property is not found, JS looks in the prototype

Inheritance – Object.setPrototypeOf(Child.prototype, Parent.prototype)

constructor.js

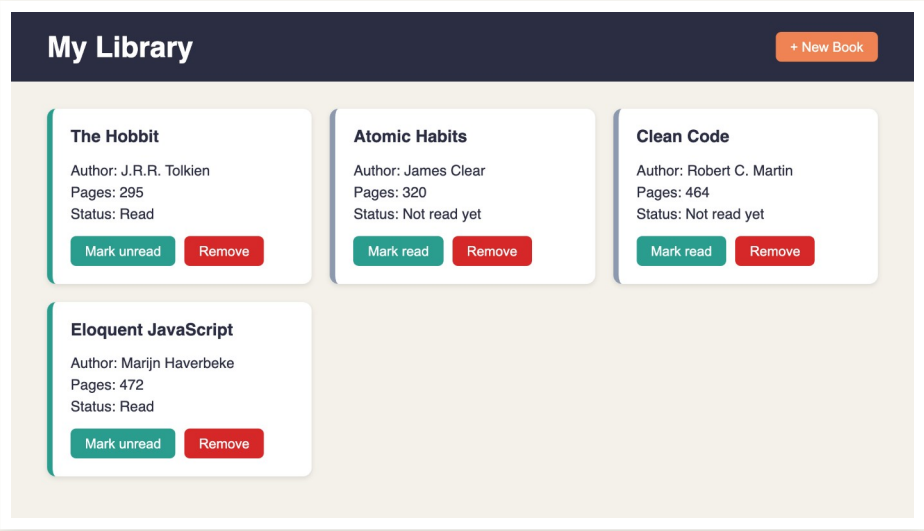
```
function Book(title, author, pages, read) {
  if (!new.target) throw Error("Use new");
  this.title = title;
  this.author = author;
  this.pages = pages;
  this.read = read;
}

// one shared function for all books
Book.prototype.toggleRead = function () {
  this.read = !this.read;
};

const hobbit = new Book("The Hobbit",
  "Tolkien", 295, true);
hobbit.toggleRead(); // read = false
```

05

Project: Library



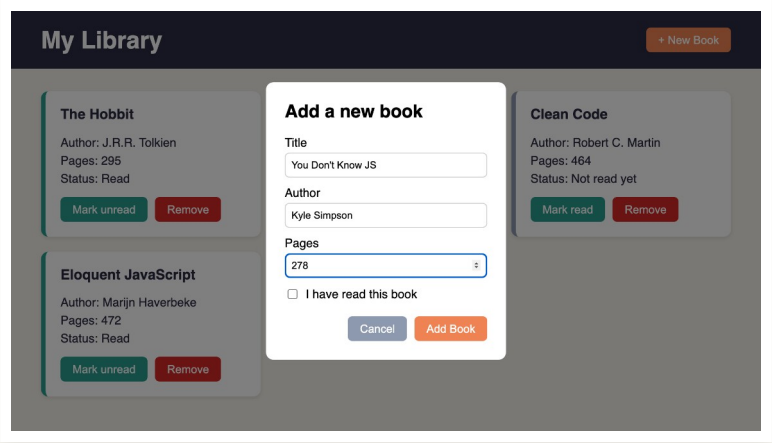
- Shows all books as cards from the myLibrary array
- "New Book" opens a <dialog> with a form
- Remove a book with its button
- Toggle read / not read with a prototype method
- Every book has a unique id from crypto.randomUUID()

Live ayxan1110.github.io/odin-library/

Code github.com/Ayxan1110/odin-library

06

Library: how I built it



The form inside <dialog>, opened with showModal()

1 **Open**
Click on "New Book" calls dialog.showModal()

2 **Submit**
preventDefault() stops the page reload, then I read the input values

3 **Store**
addBookToLibrary() creates new Book(...) and pushes it to myLibrary

4 **Display**
displayBooks() clears the container and draws a card for every book

5 **Connect**
Each card has data-id, so Remove finds the right book in the array

07

Factory functions and the module pattern

Scope – variables live inside the function or block where they are declared

Closure – an inner function remembers the variables of its outer function

Factory function – a function that returns an object, no new and no this needed

Private variables – only reachable through the returned functions

Module (IIFE) – a factory that runs one time, for things we need only once

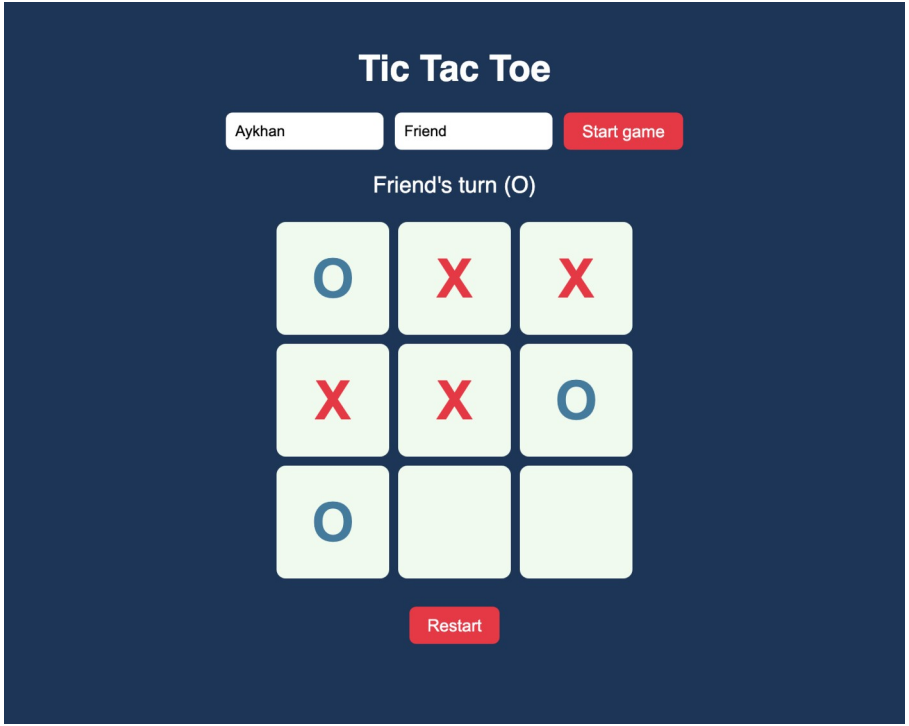
factory.js

```
// factory: we can make many players
function createPlayer(name, mark) {
  return { name, mark };
}
const p1 = createPlayer("Aykhan", "X");

// module: runs once, board stays private
const Gameboard = (function () {
  const board = ["", "", "", "", "",
                 "", "", "", ""];
  const getBoard = () => board;
  const placeMark = (i, mark) => {
    if (board[i] !== "") return false;
    board[i] = mark;
    return true;
  };
  return { getBoard, placeMark };
})();
```

08

Project: Tic Tac Toe



- Two players enter their names
- Click a cell to place X or O
- Taken cells cannot be clicked again
- Detects all 8 winning lines and a tie
- Restart button starts a new round

Live ayxan1110.github.io/odin-tic-tac-toe/

Code github.com/Ayxan1110/odin-tic-tac-toe

09

Tic Tac Toe: code structure

`Gameboard`

module · IIFE

Private board array.
getBoard(), placeMark(), reset()

`createPlayer()`

factory function

Returns { name, mark }.
Called twice: X and O

`GameController`

module · IIFE

Whose turn it is, win and tie check.
start(), playRound(index)

`DisplayController`

module · IIFE

The only part that uses the DOM.
render(), click and form events

Game logic works without the DOM, so it can be played in the console

10

Problems I solved

form.title was empty

A <form> already has a built-in title property, so form.title did not return my input. I used document.querySelector("#title") instead.

Books were duplicated

Every time I added a book, all cards were added again. displayBooks() now clears the container first and then draws the array.

Win on the last move

If someone wins with the 9th move, the game must not say "tie". I check for a winner first and only then for a full board.

Nine click listeners

Instead of adding a listener to every cell, I add one listener on the board and read data-index from the clicked cell (event delegation).

11 Projects on GitHub

Library	ayxan1110.github.io/odin-library/	github.com/Ayxan1110/odin-library
Tic Tac Toe	ayxan1110.github.io/odin-tic-tac-toe/	github.com/Ayxan1110/odin-tic-tac-toe

Thank you!